OPEN ACCESS

PRZEGLĄD ELEKTROTECHNICZNY

Adam MASTALERZ[1], Tomasz SKOWRON[2], Adam DOMAŃSKI[3], Jakub SZYGUŁA[4]
ORCID: [1]0009-0004-3199-069X; [2]0009-0004-7356-543X; [3]0000-0002-9452-8361; [4]0000-0002-5728-6105
DOI: 10.15199/48.2025.08.21

The impact of diverse, alternative representations of textual input data on the quality and computational demands of selected classification models in datasets containing specialized vocabulary

Wpływ różnorodnych, alternatywnych reprezentacji danych wejściowych tekstowych na jakość i wymagania obliczeniowe wybranych modeli klasyfikacyjnych w zbiorach danych zawierających słownictwo specjalistyczne

[1] mgr inż. Adam Mastalerz, Silesian University of Technology, Faculty of Automatic Control, Electronics and Computer Science, Department of Distributed Systems and Informatic Devices, Akademicka 16 44-100 Gliwice, E-mail: adam.mastalerz@polsl.pl
[2] mgr inż. Tomasz Skowron, Silesian University of Technology, Faculty of Automatic Control, Electronics and Computer Science, Department of Distributed Systems and Informatic Devices, Akademicka 16 44-100 Gliwice, E-mail: tomasz.skowron@polsl.pl
[3] dr hab. inż. Adam Domański, prof. PŚ., Silesian University of Technology, Faculty of Automatic Control, Electronics and Computer Science, Department of Distributed Systems and Informatic Devices, Akademicka 16 44-100 Gliwice, E-mail: adam.domanski@polsl.pl
[4] dr inż. Jakub Szyguła, Silesian University of Technology, Faculty of Automatic Control, Electronics and Computer Science, Department of Distributed Systems and Informatic Devices, Akademicka 16 44-100 Gliwice, E-mail: jakub.szygula@polsl.pl.

Correspondence address: adam.mastalerz@polsl.pl

Abstract: This paper investigates the impact of various representations of textual input data on the quality of selected classification models in datasets containing specialized vocabulary. By employing various vectorization techniques and machine learning models, the study aims to improve our understanding of the relationship between textual representations and model performance and efficiency.

Streszczenie: W tym artykule zbadano wpływ różnych reprezentacji tekstowych danych wejściowych na jakość wybranych modeli klasyfikacji w zestawach danych zawierających specjalistyczne słownictwo. Poprzez zastosowanie różnych technik wektoryzacji i modeli uczenia maszynowego, badanie ma na celu poprawę naszego zrozumienia relacji między reprezentacjami tekstowymi, a wydajnością i efektywnością procesu uczenia.

Keywords: Natural Language Processing; Classification Models; Vectorization Techniques; Sustainable AI; Explainable AI

Słowa kluczowe: Przetwarzanie Języka Naturalnego; Modele Klasyfikacyjne; Techniki Wektoryzacji; Zrównoważone AI; Wyjaśnialne AI

Introduction

In the rapidly advancing domain of natural language processing (NLP), the need for accurate and efficient text classification becomes increasingly important. Large volumes of text require models that can interpret, classify, and use this information effectively. NLP plays a key role across many sectors. It supports systems such as search engines, recommendation tools, sentiment analysis, and automated content moderation. [16].

Central to the evolution of NLP is the exploration of diverse textual representations and their impact on the performance of classification models. Particularly when dealing with specialized or domain-specific vocabularies. The development of vectorization techniques such as Word2Vec, GloVe, and BERT has marked a paradigm shift in how machines understand and process language, enabling a deeper grasp of semantic relationships and contextual nuances [17].

This paper examines how different text representations affect classification performance. It highlights the usefulness of simpler models when combined with strong vectorisation techniques. In a field often driven by complex algorithms, these simpler models can still perform well. When supported by advanced text embeddings, they often handle specialised vocabulary effectively. This challenges the common view that greater complexity always leads to better results in NLP [18].

Our investigation sheds some light on the comparative performance of various classification models, from traditional algorithms to state-of-the-art neural networks, across different vectorization techniques on a selected dataset with a specialized vocabulary. By systematically evaluating the interplay between model simplicity and the sophistication of textual representations, we aim to uncover insights that could guide the development of more efficient NLP tools based on simpler models [19].

This study offers insights that go beyond academic interest. The results can help improve NLP applications in many industries. We recommend using models that balance accuracy with simplicity. This supports a sustainable approach that values efficiency, interpretability, and adaptability to new data or domains [20].

Related works

Recent advances in Artificial Intelligence have led to its application across diverse domains, such as CV analysis [26] and optimizing sampling strategies in aerospace materials testing [27]. A critical area of research is the representation of textual input data, especially for tasks involving specialized vocabulary.

Foundational embedding techniques include Word2Vec [1,21] and GloVe [2], which capture semantic relationships between words using vector representations. Subsequent models such as FastText [6,22] improved handling of rare and out-of-vocabulary terms through subword embeddings. Contextual embeddings, introduced by models like ELMo [7,23] and BERT [3], further advanced performance by incorporating sentence-level context. Notably, Sennrich et al. [4] demonstrated the effectiveness of subword-level representations in translation tasks, while Gaspers et al. [5] emphasized the need for domain adaptation in classification.

Transfer learning has also yielded significant progress. ULMFiT [8,24] and domain-adaptive pretraining [9,25] enable models to specialize in particular linguistic contexts. Multilingual and cross-lingual approaches, such as XLM [10] and T5 [11], enhance model flexibility across languages. GPT-3 [12] demonstrated remarkable zero- and few-shot performance across tasks, highlighting the power of general-purpose pretrained models.

Recent comparative studies reinforce these findings. Gumińska et al. [14] evaluated embedding methods against traditional techniques, affirming the superiority of modern embeddings in domain-specific contexts. Xi Yang et al. [15] proposed a hybrid of topic modeling and transfer learning, further improving text classification for specialized vocabulary.

Collectively, these works underscore the importance of tailoring input representations to domain-specific language, a key factor in the effectiveness of modern NLP models.

Our contributions: Presented research delves into the interplay between distinct textual representations and the effectiveness of classification models when applied to datasets containing domain-specific language. Through an evaluation of different vectorization methods and machine learning algorithms, our findings serve to advance the development of more efficient natural language processing tools and facilitate better decision-making in various industries.

Research Methodology

Within this section, we are going to present the results of our experiments, involving four distinct text classification models: Multinomial Naive Bayes classifier, Logistical Regression, Convolutional Neural Networks, Recurrent Neural Network with Long-Short Term Memory and Transformer Encoder-based Neural Network. Two of which can be described as rather simple, traditional, easily explainable models for that problem, and two representing black box solutions that have gained popularity in recent years.

Each of the models, except for the Transformer Encoder-based models, was tested in three scenarios:

- text data is represented in the simplest form that allows the model to perform its duties
- text data is represented in the form of word vectors provided by the Word2Vec solution trained on the GoogleNews dataset containing common, everyday vocabulary
- text data is represented in the form of paragraph vectors provided by the Doc2Vec solution trained on our dataset containing a specialized vocabulary.

A Transformer Encoder-based architecture was employed exclusively for textual data represented in a simple tokenized format. In lieu of the Word2Vec representation, a trainable Embedding Layer was utilized, enabling the model to learn contextually informed token representations. Notably, the application of

Doc2Vec results in an output wherein sequences of tokens are replaced by n-dimensional vectors encapsulating the broader contextual meaning of a document. Consequently, the Transformer Encoder layer is rendered inapplicable to such data, as its primary utility lies in capturing relationships within sequential data and preserving positional dependencies.

The dataset used for the purpose of this research was obtained through standard web scraping procedures of StackOverflow articles available to the public. The raw version of the dataset contained 10286120 words, including HTML markdowns and code snippets.

The preprocessing steps included:

- removal of whitespace characters
- removal of punctuation characters
- removal of stopwords
- removal of HTML markdown
- undersampling of majority classes to achieve uniform distribution of classes within the dataset.

Preprocessing steps reduced the dataset to one-third of its original size and ensured that it is balanced (figure 1). The dataset contains twenty classes containing problems posted by StackOverflow's users from the domains of twenty different programming languages.

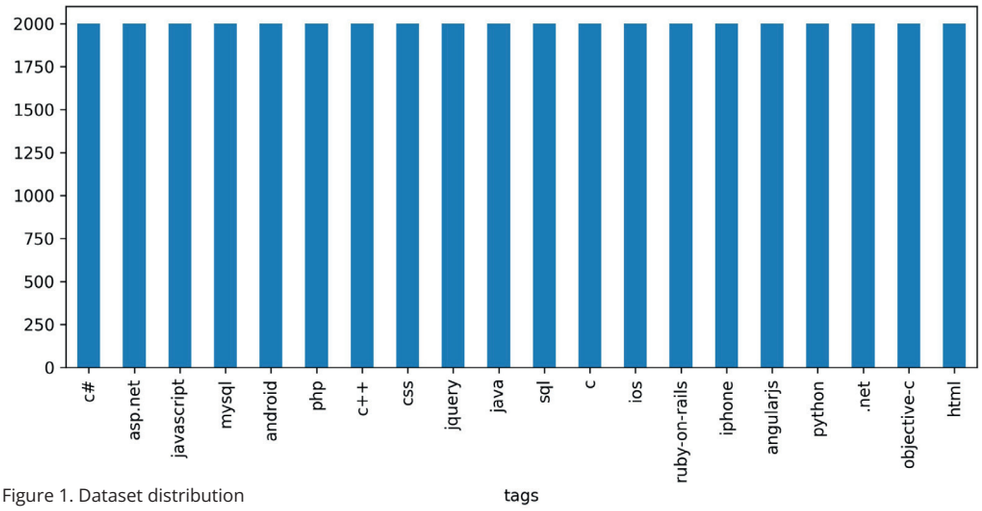

Figure 1. Dataset distribution

Multinomial Naive Bayes Classifier

To be able to use a Multinomial Naive Bayes classifier, we need to use any form of text vectorization to obtain a numerical representation of the input data.

To make the study meaningful, we will use the simplest form of vectorization, which is count vectorization. It works by counting the occurrences of certain words or n-grams from the dictionary and taking them as the successive coordinates of a vector representing the given sentence/text/document/line of text/et cetera. Obtained results are presented in Figure 2.

The resulting vector does not express any deeper semantic dependencies, but it is sufficient for the proper functioning of a Multinomial Naive Bayes classifier. They are quite good for such a simple data preparation and model. However, there is still some room for improvement. The model (figure 2) primarily has a problem with confusing posts about SQL with posts about C, posts about HTML with posts about C++, and posts about iOS with posts about Ruby on Rails.

As we can see, using the Word2Vec representation did not help the Multinomial Naive Bayes Classifier achieve better performance. Same code, same data, different representation, and the result obtained is completely different. The model (figure 3) was not able to learn effectively. An additional step of MinMaxScaler was necessary for the model to run at all because the Multinomial Naive Bayes classifier does not accept vectors with negative values, which can occur in Word2Vec output. Additionally, on Figure 3 it is visible that misclassification often resulted in predicted label equal to "php". This suggests that php language contains the most diverse keywords which in conjunction with precalculated Word2Vec representations fit into most languages.

The input data in the form of embeddings resulting from the Doc2Vec model were accepted without any additional modifications to the Multinomial Naive Bayes classifier. The performance of the model (figure 4) is noticeably better; the misclassification cases happen much more rarely.

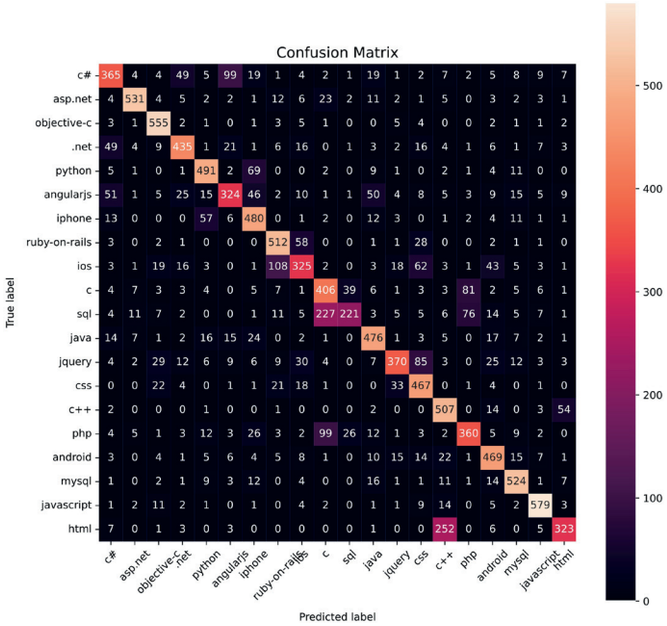

Figure 2. Multinomial Naive Bayes classifier with standard representation

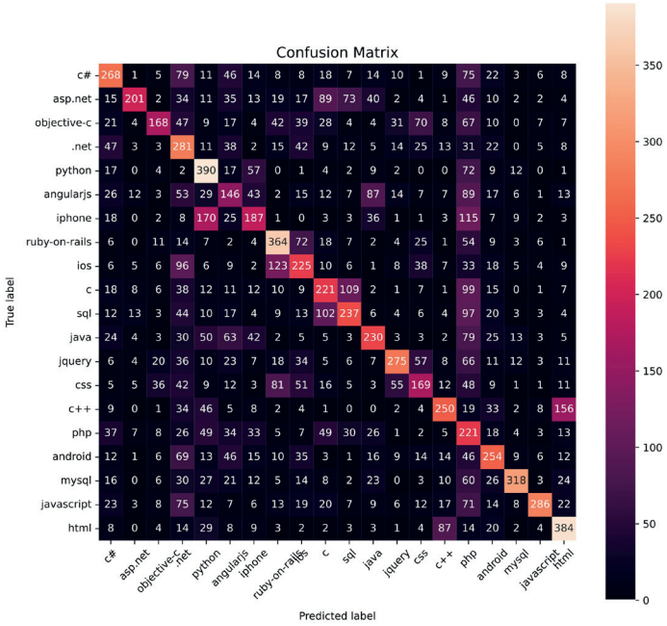

Figure 3. Multinomial Naive Bayes classifier with Word2Vec data representation

Logisitic Regression

Logistic regression is a commonly used technique for binary classification, and it requires some form of a numerical representation of the input data. This representation is obtained by vectorizing the text, using a method such as Count Vectorizer. The experimental results obtained with this model are better than the ones of the Multinomial Naive Bayes classifier, as the model's ability to distinguish between posts on C and SQL has significantly improved, and the accuracy for the remaining classes is maintained. The number of areas in which it makes mistakes is narrowed, and in those areas, it makes incorrect predictions much less frequently (figure 5).

Incorporating an alternative form of text representation provided by Word2Vec trained on the GoogleNews dataset containing everyday vocabulary did not lead to an enhance- ment in the performance of logistic regression (figure 6). Nevertheless, the model was able to learn to recognize the designated classes with reasonable precision. This outcome attests to the effectiveness of CountVectorizer in producing a reliable numerical representation of the input data.

In the case of logistic regression, the data representation resulting from Doc2Vec proved to be helpful. The model was able to significantly improve compared to the "normal" representation resulting from Word2Vec (figure 7).

Logistic regression achieved even better performance with input data representation provided by Doc2Vec, a distributed memory-based model that produces paragraph vectors. The model was able to improve significantly compared to the "standard" representation and the representation obtained through Word2Vec. The logistic regression algorithm readily accepted the input data in the form of vectors obtained from the Doc2Vec model without requiring any additional modifications.

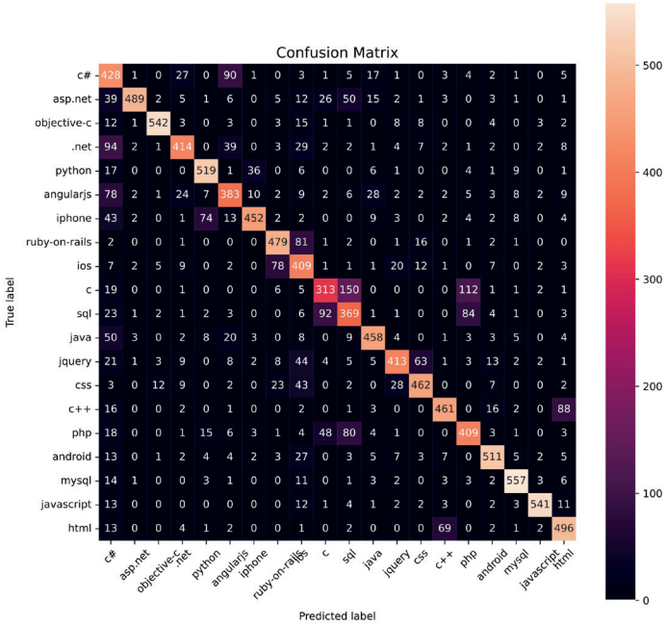

Figure 4. Multinomial Naive Bayes classifier with Doc2Vec data representation

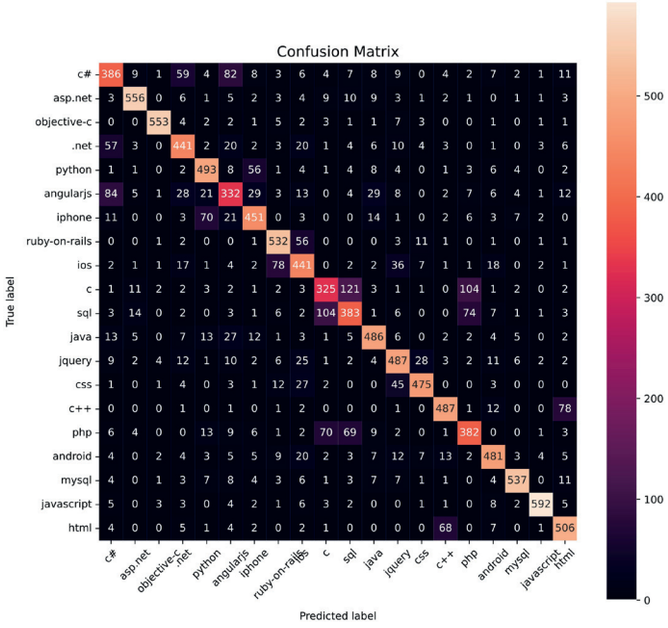

Figure 5. Logistic Regression with standard data representation

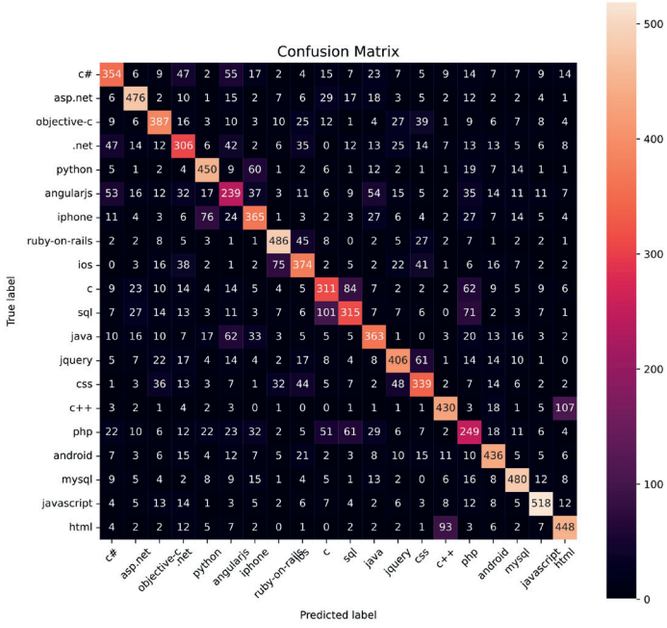

Figure 6. Logistic Regression with Word2Vec data representation

Convolutional Neural Network

The first neural network model tested in this study is the Convolutional Neural Network (CNN). To prepare the data for proper loading, it will undergo tokenization and conversion into sequences of vectors. The tokenization was achieved using Tokenizer class from tensorflow.keras.preprocessing.text package.

The next step was to create the network itself. The first layer employed was the embedding layer, which transforms positive integers into dense vectors of a specific size. The size of embedding used in this research was 100, while maximum number of words used was set to 50.000. The next layer was the SpatialDropout1D layer, which applies a dropout mask to the input and output at each calculation step, utilizing a variance-based inference approach. The Conv1D convolutional layer was added on top of SpatialDropout1D, creating a kernel associated with one dimension of the data and assisting in generating the output tensor. In this research there was only one Conv1d layer which used 200 filters and kernel size set to 20. Values calculated by Conv1D were later passed to GlobalMaxPooling1D and finally passed through a Dense layer with 'softmax' activation.

The network was then designed, compiled, and trained on the prepared data in the simplest acceptable form, producing the following results (figure 8).

The alternative approach, using word2vec the tokenized was applied later for performance comparison. Data preprocessing and adjustments were necessary. Tokenized input sequences (code samples) were subject to conversion using Word2Vec from tokens to vectors thus yielding a training set with 3 dimensions. From each of these vectors, representing a single token in the sequence in multi-dimensional space, an average was calculated. Such data was then passed to the same CNN network architecture. This technique delivered worse results than standard data representation (figure 9).

The CNN model was then successfully adapted to input data in the form of embeddings obtained by the Doc2Vec method. This approach was implemented by training Doc2Vec on the input data and converting the input sequences to (document) vectors. Such vectors were treated as pretrained embeddings in the CNN model. AThese representations were lightweight enough to allow experiments to be conducted, but the results obtained demonstrated that more advanced models are capable of producing better representations through the use of an embedding layer which is a standard part of their architecture.

As we can see on a confusion matrix (figure 10), the general patterns characterizing different classes were learnt correctly, but using a non-optimal form of representation for the given architecture resulted in a model over-focusing on some classes (for example "angularjs" class) and searching for them in places that they simply weren't present, which resulted in a worse overall performance. Notably, similar behaviour to one from NaiveBayes classifier used with Word2Vec representation can be observed on Figure 10, but here the Doc2Vec was an element that yields vectors that suggest misclassification of 'javascript' and 'c#' samples.

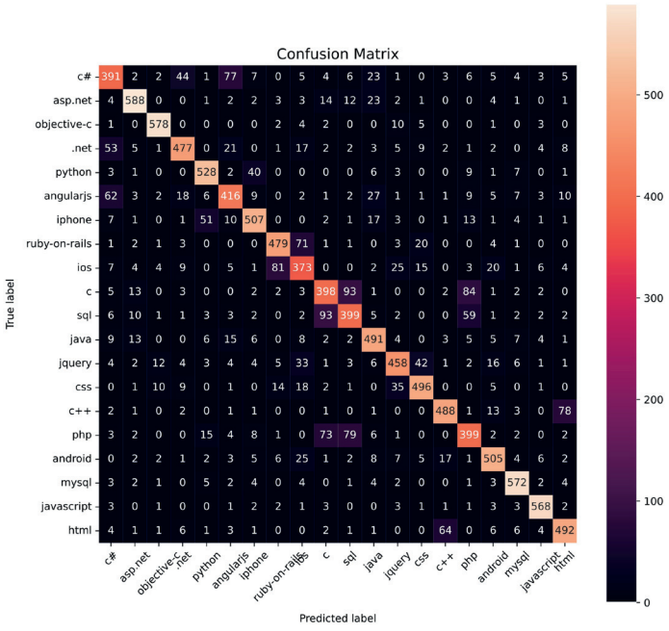

Figure 7. Logistic Regression with Doc2Vec data representation

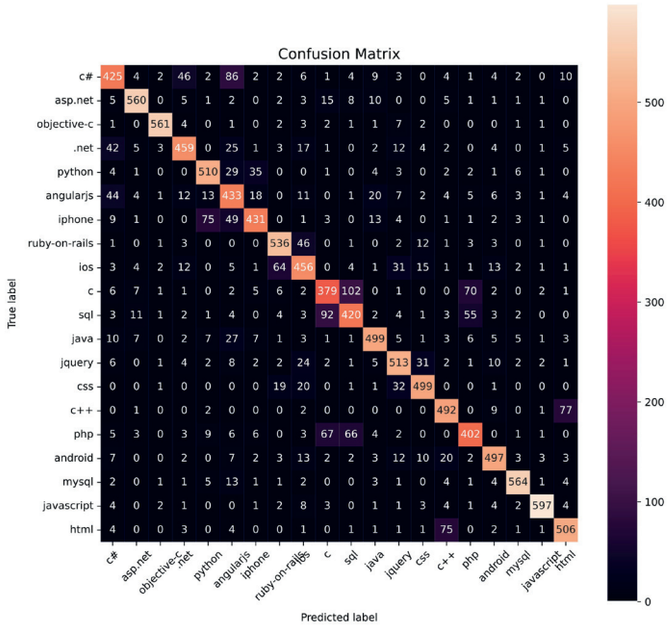

Figure 8. Convolutional Neural Network with standard data representation

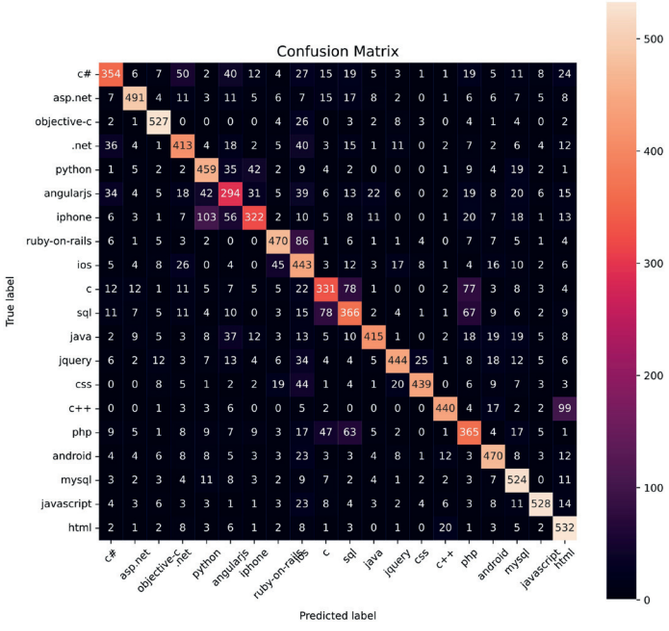

Figure 9. Convolutional Neural Network with Word2Vec data representation

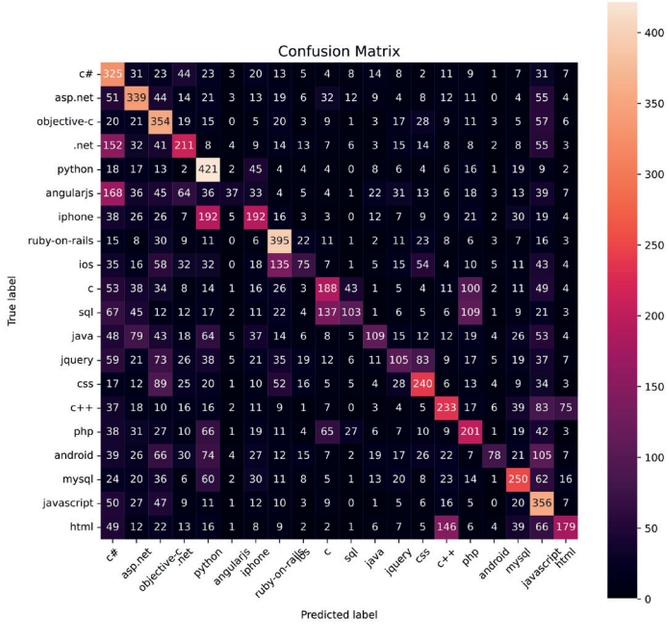

Figure 10. Convolutional Neural Network with Doc2Vec data representation

Recurrent Neural Network with Long--Short Term Memory

The second neural network model tested in this study is the Recurrent Neural Network with Long--Short Term Memory (LSTM). In the architecture of this approach, the convolutional layer is replaced by the LSTM layer, which helps the network capture long-term dependencies between analysed examples. Results are presented in Figure 11.

In the case of Word2Vec, our efforts were insufficient to adjust the data properly to the model and its characteristics which would yield satisfactory results.

Fortunately, it was possible to successfully adapt LSTM's architecture to the use of input data in the form of embeddings obtained by the Doc2Vec method. This was achieved by passing the data processed identically as for CNN with Doc2Vec approach. These representations were lightweight enough to

allow experiments to be conducted, but heavy enough to significantly extend the training time compared to other tested models.

The results obtained demonstrate that more advanced models are capable of producing better representations through the use of an embedded embedding layer.

Figure 12. Recurrent Neural Network with Long--Short Term Memory with Doc2Vec data representation

As we can see on a confusion matrix (figure 12), the general patterns characterizing different classes were learnt correctly, but using a suboptimal form of representation for the given architecture resulted in a model over-focusing on some classes (for example "angularjs" class) and searching for them in places that they simply weren't present, which resulted in a worse overall performance. The same issue baffled other neural network models tested in this paper.

Transformer Encoder-based Neural Network

The third neural network architecture (figure 13) under evaluation was the Transformer Encoder. As a sequence-to-sequence model, the Transformer Encoder necessitates specific modifications for its effective application in sequence classification tasks. Notably, its input must be vectorized, as raw tokenized data may hinder the model's ability to capture the semantic meaning of a sequence. To address this, a common approach involves incorporating a trainable embedding layer prior to processing the data through the Transformer Encoder.

Furthermore, in the context of sequence classification, the architecture is augmented with a trainable parameter, which is appended to the input data array. The dimensionality of this parameter aligns with the batch size and the length of the vector representing an individual token. This parameter is concatenated with the output of the embedding layer and subsequently passed through the Transformer Encoder. As a result, the Transformer Encoder's output sequence retains information from this trainable parameter, specifically within the first vector of the output sequence. Finally, this vector undergoes processing via standard linear layers and normalization, ultimately yielding an output interpretable as the sequence classification result.

The results obtained using the neural network based on the transformer encoder outperformed all other models (figure 14). The model achieved accuracy of 95,2% on validation Dataset (table 1.).

Training and Inference Time

The pursuit of sustainable AI necessitates a rigorous analysis of energy consumption and strategies for its optimisation. In this study, the authors assessed the performance of the applied models in terms of both training and inference times. The results are included in Table 1. where best (most accurate) models were highlighted with gold (1st place), silver (2nd place), bronze (3rd place) _colours. Additionally Naive Bayes model was marked with green as the model with the best ratio of inference and training time to accuracy achieved. While absolute measurements of time or po-

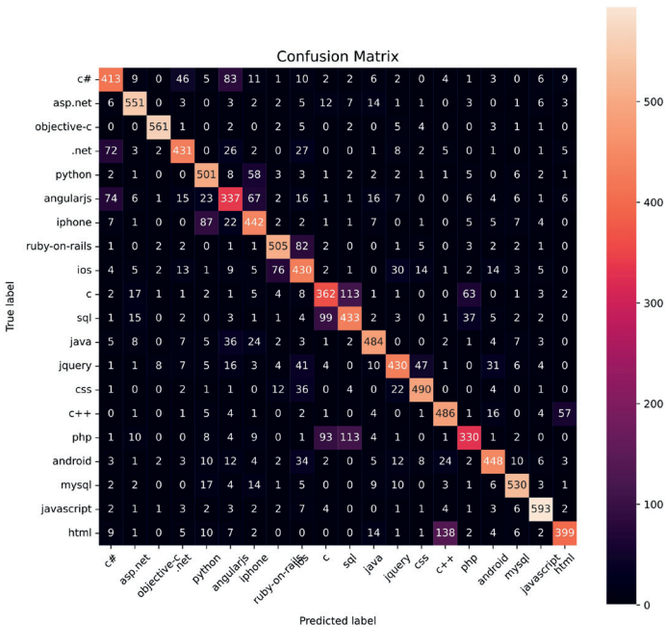

Figure 11. Recurrent Neural Network with Long-Short Term Memory with standard data representation

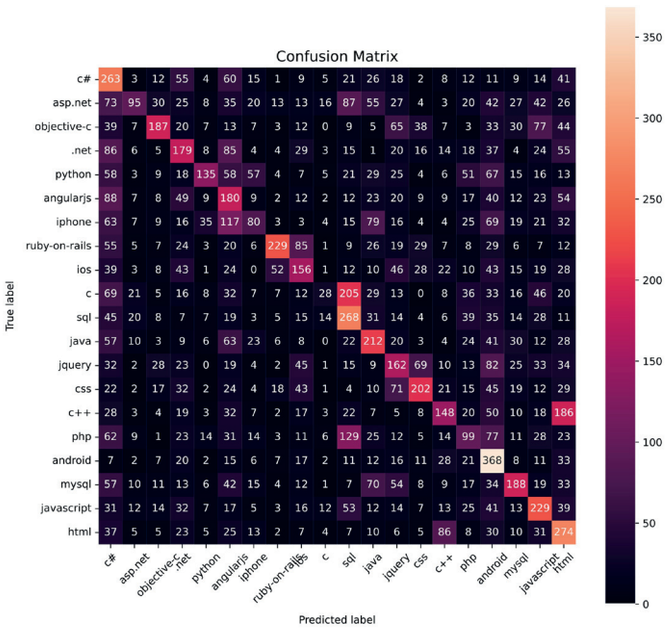

Figure 12. Recurrent Neural Network with Long-Short Term Memory with Doc2Vec data representation

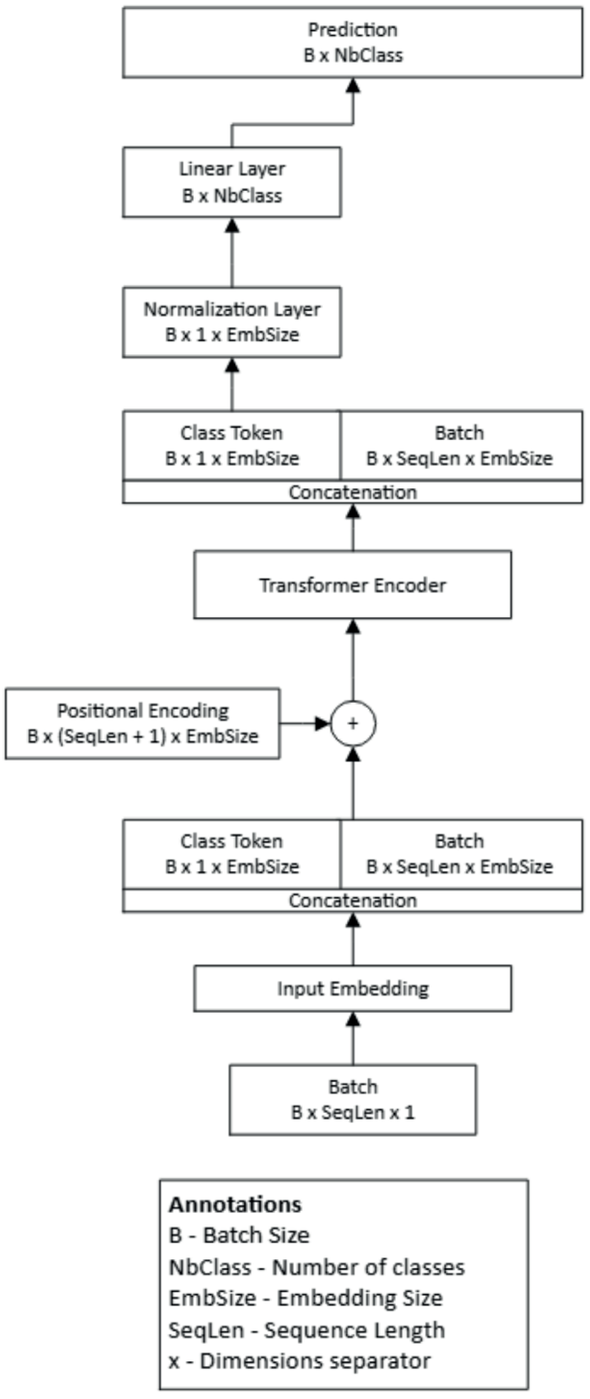

Figure 13. Applied Transformer Encoder-based Neural Network Architecture for sequence classification

wer consumption are inherently hardware-dependent and will vary across different computational environments, we conducted all of our evaluations on a standardised hardware. This allows us to capture the relative proportions of resource usage across all models and to provide a general understanding of the relationship between architectural complexity and computational demands.

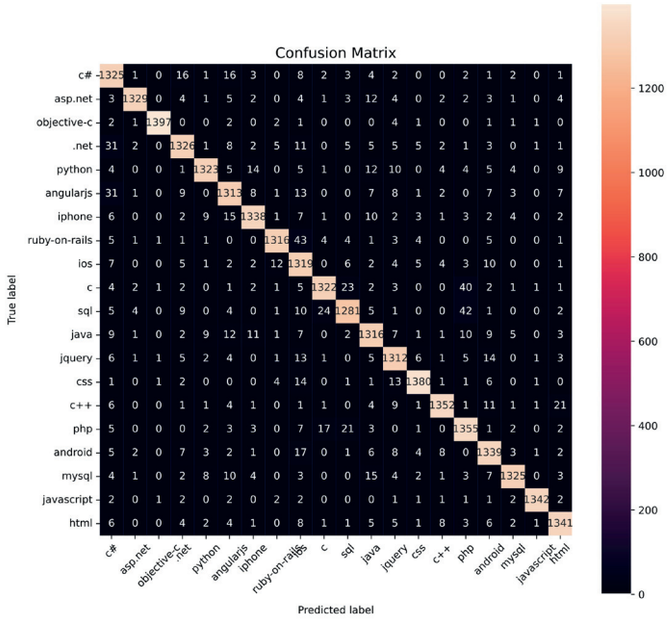

Figure 14. Transformer Encoder-based Neural Network

While simpler architectures, when combined with appropriate data representations, may not achieve absolute state-of-the-art performance, their results remain competitive. This makes them a compelling choice, particularly given their substantial advantages in terms of explainability, training efficiency and energy consumption. Moreover, inference time–a critical factor in the long-term operational costs of deployed systems–sees marked improvements with these models, further justifying their adoption in resource-conscious applications.

Table 1. Training and Inference time

Model	Epochs	Training Time per epoch (s)	Inference Time (s)
Naive Bayes	1	3,26	0,000051
Logistic Regression	1	107,90	0,000062
CNN	5	87,85	0,000830
LSTM	14	113,41	0,002330
Naive Bayes (Word2Vec)	1	0,12	0,000002
Logistic Regression (Word2Vec)	1	24,22	0,000001
CNN (Word2Vec)	5	144,50	0,002266
Doc2Vec Training (Doc2Vec)	30	4,56	n/a
Naive Bayes (Doc2Vec)	1	0,13	0,000003
Logistic Regression (Doc2Vec)	1	10,53	0,000001
CNN (Doc2Vec)	10	8,85	0,000168
LSTM (Doc2Vec)	100	49,04	0,000028
Transformer Encoder	250	448,35	0,008522

Conclusions

The broad field of data analysis, and in particular, the natural language processing techniques discussed in this work, holds immense promise for large-scale organizations. The consensus among experts is that the potential benefits extend beyond the service sector, enabling the delivery of wider, cheaper, and more effective services in the public domain as well. However, organizations that work with textual data sets, especially those containing specialized vocabulary, must go through the arduous process of selecting the appropriate tools to solve a variety of problems, which are presently addressed manually. It is, therefore, crucial to not only choose the right techniques but also to tailor them to the specific nature of the documents being processed.

The results obtained from highly diversified data sets can be surprising, with simpler models sometimes outperforming far more complex ones, requiring only a different representation of the data. However, there is no one-size-fits-all solution that works for every algorithm and every data set. The same data, represented in different ways, may be far more "readable" for Algorithm A but may be particularly incompatible with the way Algorithm B interprets input data. Representations created by models (or model layers) trained on data that differs from that being analysed can significantly downgrade the quality of models that perform very well on "clean" data. In our experiments, we used four models that employ data in three forms:

- the simplest form in which the model can operate on
- the Word2Vec embedding proposed by the model, trained on the Google News data set containing commonly used, everyday vocabulary
- the Doc2Vec embedding proposed by the model, trained on the analysed data set containing specialized vocabulary

The aggregate results of the experiments are presented in the Table 2. where best (most accurate) models were highlighted with gold (1st place), silver (2nd place), bronze (3rd place) colours. Additionally Naive Bayes model was marked with green as the model with the best ratio of inference and training time to accuracy achieved.

The first salient conclusion drawn from the experiments is that models employing Word2Vec representations do not perform optimally when applied to datasets of a more specialized nature than the corpora on which the embeddings were trained. Since Word2Vec was originally developed using generic text, its representations fail to capture domain-specific nuances, leading to the introduction of noise or irrelevant features. This shortcoming is reflected in the consistently negative impact on performance metrics across all tested models.

Table 2. Models' Performance

Model	Accuracy	Precision	Sensitivity	F1–Score
Multinomial Naive Bayes classifier	72,7%	73,2%	72,7%	72,0%
Multinomial Naive Bayes classifier	42,2%	47,3%	42,2%	43,3%
+ Word2Vec data representation	–30,5%	–25,9%	–30,5%	–28,7%
Multinomial Naive Bayes classifier	75,8%	77,4%	75,8%	76,2%
+ Doc2Vec data representation	3,1%	4,2%	3,1%	4,2%
Logistic Regression	77,7%	77,7%	77,6%	77,6%
Logistic Regression	64,4%	64,1%	64,2%	64,2%
+ Word2Vec data representation	–13,3%	–13,6%	–13,4%	–13,4%
Logistic Regression	80,0%	79,8%	79,8%	79,8%
+ Doc2Vec data representation	2,3%	2,1%	2,2%	2,2%
Convolutional Neural Network	81,1%	81,3%	81,1%	81,1%
Convolutional Neural Network	71,8%	73,0%	71,8%	71,9%
+ Word2Vec data representation	–9,3%	–8,3%	–9,3%	–9,2%
Convolutional Neural Network	36,6%	39,7%	36,4%	34,3%
+ Doc2Vec data representation	–44,5%	–41,6%	–44,7%	–46,8%
Recurrent Neural Network with LSTM	76,3%	76,6%	76,6%	76,2%
Recurrent Neural Network with LSTM	5,0%	5,0%	0,0%	0,0%
+ Word2Vec data representation	–71,3%	–71,6%	–76,6%	–76,2%
Recurrent Neural Network with LSTM	30,6%	33,3%	30,8%	29,9%
+ Doc2Vec data representation	–45,7%	–43,3%	–45,8%	–46,3%
Transformer Encoder-based Neural Network	95,2%	95,3%	95,2%	95,2%

By contrast, Doc2Vec representations trained directly on the target dataset had a notably positive effect, particularly for simpler models. The conventional Multinomial Naive Bayes classifier, for instance, performed on par with the LSTM-based recurrent neural network–an architecture typically regarded as one of the most effective in classification tasks, second only to transformer-based models, which were also successfully applied in this context. Logistic regression, when paired with Doc2Vec embeddings, even outperformed the LSTM and closely matched the second-best model. This is likely due to the characteristics of the dataset, where small sample sizes favoured local pattern detection over sequential memory. These findings suggest that investing in more complex architectures may not always yield proportional performance gains and highlight the efficiency and cost-effectiveness of sim-

pler models–particularly when enhanced with task-specific representations.

Additionally, it is worth highlighting that simpler models that this research proved to be competitively effective and as such they offer environmental benefits. Less computationally intensive models consume significantly less energy, thereby reducing the carbon footprint associated with Big Data analysis. When performance is comparable, preferring models with lower resource requirements can help mitigate the environmental costs of running large-scale datacentres.

The analysis in this study was limited to a single dataset, which does not prove identical effectiveness in general. However, as long as given dataset contains certain keywords that distinguish various classes, such models should be capable of learning such relationships. Even models performing relatively simple mathematical operations (e.g., Naive Bayes, Logistic Regression) can achieve impressive results. For some domains, where the keywords are less specific, or define the class of a given sequence only when in conjunction with other tokens, CNN and LSTM models could possibly detect such patterns (co-occurrences) better. The transformer-based architecture model will deliver results with less dependency on the dataset or domain characteristics, at the price of performance and training time. It is then up to stakeholders to select the most suitable approach.

A further key observation is that none of the externally pre-trained embedding models outperformed internal embedding layers. This underscores the value of representations generated within the neural network architecture itself, which are inherently aligned with the model's structure and can better adapt to dataset-specific features. As Montesquieu might have put it: Le mieux est l'ennemi du bien.

Another important conclusion is the advantage of training representations on data with characteristics closely related to the target domain. Such an approach facilitates the learning of relevant features and relationships that might be overlooked by embeddings derived from general-purpose data.

Finally, the experiments indicate that well-tuned representations are particularly beneficial for simpler models that do not generate internal embeddings. With adequate external support, these models can rival more complex architectures, offering a viable solution in contexts where interpretability or resource constraints rule out black-box approaches like neural networks. While transformer-based models remain unmatched in sheer performance, their resource demands are orders of magnitude higher than those of their simpler competitors.

Conclusions – Quick Summary:

1. Pretrained Word2Vec embeddings trained on generic corpora perform poorly on specialized datasets, no matter the architecture.
2. Doc2Vec embeddings, trained on the target data, enhance the performance of simple models like Naive Bayes and Logistic Regression.
3. Transformer-based models yield the best results but come at a significantly higher computational cost.
4. Simpler models, when paired with appropriate representations, offer a balance between performance, interpretability, and energy efficiency and can compete with more complex models.

Received: 06.04.2025, **Accepted:** 05.08.2025, **Published:** 29.08.2025

REFERENCES

[1] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, J. Dean, Distributed representations of words and phrases and their compositionality, *Advances in Neural Information Processing Systems*, (2013), Vol. 26, pp. 3111–3119

[2] J. Pennington, R. Socher, C. D. Manning, GloVe: Global Vectors for Word Representation, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, (2014), pp. 1532–1543

[3] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, (2018), pp. 4171–4186

[4] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, (2016), Volume 1: Long Papers, pp. 1715–1725

[5] J. Gaspers, Q. Do, T. Röding, M. Bradford, The impact of domain-specific representations on BERT-based multi-domain spoken language understanding. In *Proceedings of the Second Workshop on Domain Adaptation for NLP*, (2021), pp. 28–32,

[6] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, Enriching Word Vectors with Subword Information, *Transactions of the Association for Computational Linguistics*, (2017) 5, 135–146

[7] M. E. Peters et al., Deep contextualized word representations, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, (2018), Volume 1 (Long Papers), pp. 2227–2237

[8] J. Howard and S. Ruder, Universal Language Model Fine-tuning for Text Classification, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics,* (2018), Volume 1: Long Papers, pp. 328–339

[9] S. Gururangan et al., Don't Stop Pretraining: *Adapt Language Models to Domains and Tasks, In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, (2020), pp. 8342–8360

[10] A. Conneau et al., Unsupervised Cross-lingual Representation Learning at Scale, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, (2020), pp. 8440–8451

[11] C. Raffel et al., Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer, *Journal of Machine Learning Research*, (2020), vol. 21, no. 140, pp. 1–67

[12] T. B. Brown et al., Language Models are Few-Shot Learners, *Proceedings of the 34th International Conference on Neural Information Processing Systems*, (2020), Article 159, pp. 1877–1901

[13] T. Emmanuel et al., A survey on missing data in machine learning, *Journal of Big Data*, (2021), vol. 8, no. 140

[14] U. Gumińska, A. Poniszewska-Maranda, J. Ochelska-Mierzejewska, "Systematic Comparison of Vectorization Methods in Classification Context, " Applied Sciences, (2022), vol. 12, no. 10, article 5119

[15] X. Yang, K. Yang, T. Cui, M. Chen, L. He, A Study of Text Vectorization Method Combining Topic Model and Transfer Learning, *Processes*, (2022), vol. 10, no. 2, article 350

[16] H. Zou and K. Xiang, "Sentiment Classification Method Based on Blending of Emoticons and Short Texts, *Entropy*, (2022), vol. 24, no. 3, article 398

[17] J. Zhang, F. Liu, W. Xu, and H. Yu, Feature Fusion Text Classification Model Combining CNN and BiGRU with Multi-Attention Mechanism, *Future Internet*, (2019). vol. 11, no. 11, article 237

[18] Z. Huo, Y. Fan, and Y. Huang, A Communication-Efficient Federated Text Classification Method Based on Parameter Pruning, *Mathematics*, (2023), vol. 11, no. 13, article 2804

[19] J. Mutinda, W. Mwangi, and G. Okeyo, Sentiment Analysis of Text Reviews Using Lexicon-Enhanced Bert Embedding (LeBERT) Model with Convolutional Neural Network, *Applied Sciences*, (2023), vol. 13, no. 3, article 1445

[20] F. Li, Y. Yin, J. Shi, X. Mao, and R. Shi, Method of Feature Reduction in Short Text Classification Based on Feature Clustering, *Applied Sciences*, (2019), vol. 9, no. 8, article 1578

[21] A. S. Alammary, BERT Models for Arabic Text Classification: A Systematic Review, *Applied Sciences*, (2022), vol. 12, no. 11, article 5720

[22] S. M. Alzanin, A. Gumaei, M. A. Haque, and A. Y. Muaad, An Optimized Arabic Multilabel Text Classification Approach Using Genetic Algorithm and Ensemble Learning, *Applied Sciences*, (2023), vol. 13, no. 18, article 10264

[23] E. D. Canedo and B. C. Mendes, Software Requirements Classification Using Machine Learning Algorithms, *Entropy*, (2020), vol. 22, no. 9, article 1057

[24] Z. E. Ekolle and R. Kohno, GenCo: A Generative Learning Model for Heterogeneous Text Classification Based on Collaborative Partial Classifications, *Applied Sciences*, (2023), vol. 13, no. 14, article 8211

[25] X. Yang, K. Yang, T. Cui, M. Chen, and L. He, A Study of Text Vectorization Method Combining Topic Model and Transfer Learning, *Processes*, (2022), vol. 10, no. 2, article 350

[26] Łępicki M., Kurek J., Efficient information extraction from resumes using small language models for SMEs based on Zero-Shot learning approach, *Przegląd Elektrotechniczny*, 3 (2025)

[27] Adamczyk W., Pawlak S., Durejko T., Łazińska M., Białecki R., Helcio R.B. Orlande, Widuch A., Polański, A methodology to determine the thermal diffusivity of additively manufactured jet engine blades based on laser-induced temperature field, Measurement, (2022), Volume 203